



Software Requirements Specification
Purulia Aim Physical Institute (PAPI) Platform
Version: 1.0 | Date: 28 September 2026 | Status: Draft for review
Project type: Public website + Student management 



Software Requirements Specification: PAPI Platform

Ready for PDF Export. Use your browser's "Print to PDF" or a Markdown-to-PDF tool to save this document.

Version: 1.0 | **Date:** 28 September 2026 | **Status:** Draft for review

Project Type: Public website + Student management system + Recruitment information platform + Community platform

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional, data, security, and architectural requirements for the PAPI platform. It is the blueprint for design, database schema, APIs, and code.

1.2 Background

Purulia Aim Physical Institute (PAPI) trains youth for physical tests of the Indian Army, Navy, Air Force, Police, RPF, CAPF, SSC GD, etc. Trainer Anup Kumar Mahato (ex-Army Havaldar) provides free physical training to the youth of Bengal.

1.3 Product Vision

Public Website + Training Management System + Student Portal + Recruitment Information Platform + Community

The platform must communicate Discipline (structure, progress), Energy (bold visuals, youth appeal), and Trust (transparent data, secure accounts).

1.4 Scope

- **In scope (V1):** Public site, membership application, authentication/RBAC, student portal, attendance, physical tests tracking, recruitment targets/updates, notifications, dashboards, CMS (news, events, gallery, blog, wall of fame), audit logs.
- **Out of scope (V1):** Native mobile apps, real-time chat, payment gateway, AI coach, live GPS tracking.

2. Overall Description

2.1 Operating Environment

Modern browsers (Chrome-first; optimized for Android mobile data). Responsive from 360px upward. Server: Node.js LTS, PostgreSQL 15+.

2.2 User Classes

Role	Description
Visitor	Unauthenticated; reads content, submits applications.
Applicant	Submitted application, awaiting approval.
Student	Approved member; tracks progress, attendance, targets.
Trainer	Manages batches, records attendance and test results.
Content Mgr	Manages news, events, gallery, blog moderation.
Admin	Manages users, batches, recruitments, notifications.
Super Admin	Full system access. Mandatory 2FA.

3. Architecture & Technology Stack

Style: Modular Monolith

- **Frontend:** Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui, Framer Motion, Recharts.
- **Backend:** NestJS (REST API), TypeScript, Prisma ORM.
- **Database:** PostgreSQL.

- **Storage & Comms:** Cloudinary/S3 (Media), Nodemailer/SMTP.
- **Security:** JWT (HttpOnly, Secure cookies), Argon2id password hashing, strict RBAC.

Execution Prompts for Code Generation

Below is a highly optimized, sequential prompt chain designed to be fed into an LLM context. These prompts are engineered to extract senior-level, memory-efficient, generic, and DRY (Don't Repeat Yourself) code while minimizing token usage.

Instructions for use: Copy and paste these prompts one by one into your AI coding assistant.

Prompt 1: Project Initialization & Architecture Setup

Plaintext



Act as a Senior Software Engineer (15+ years experience). We are building the PAPI Platform.
Task: Initialize the monorepo workspace.

1. Provide the terminal commands to initialize a monorepo (e.g., using npm workspaces).
 2. Provide the commands to install all core dependencies: Prisma, Zod, Tailwind, shadcn.
 3. Output the exact, highly scalable directory tree for both the 'web' and 'api' applications.
- Do not write implementation code yet. Only commands, configurations (package.json workspaces).

Prompt 2: Database Schema & Global Types

Plaintext



Act as a Senior Database Architect. Context: PAPI Platform.

Task: Write the `schema.prisma` file and shared TypeScript/Zod schemas.

1. Create a highly optimized Prisma schema covering: Users, Roles, Students, Batches
 2. Enforce strict relational integrity (foreign keys, cascading rules for soft-deletion)
 3. Write a generic Zod validation factory for pagination and sorting that can be reused
- Write DRY, highly efficient code. Avoid redundant fields.

Prompt 3: Reusable UI Components & Design System

Plaintext



Act as a Senior Frontend Engineer. Context: PAPI Platform Next.js App.

Task: Build the core generic, highly reusable UI components using Tailwind and shadow

1. Define the Tailwind theme configuration using the specific brand tokens: Deep Olive
 2. Create a generic, heavily typed `  - 3. Create a generic `
- Rule: Zero duplicate code. Make components flexible enough to handle both Admin dash

Prompt 4: Backend Security, Auth & RBAC

Plaintext

Act as a Senior Security & Backend Engineer. Context: PAPI Platform NestJS API.
Task: Implement the core Authentication and Authorization modules.

1. Write the AuthController and AuthService. Implement Argon2id password hashing.
2. Implement secure JWT authentication returning tokens strictly via 'HttpOnly', 'Secure' cookies.
3. Write a generic '@RequireRoles()' decorator and a globally applicable 'RolesGuard'.
4. Provide a robust global Exception Filter that sanitizes database error messages by removing sensitive details.

Ensure $O(1)$ or $O(\log n)$ time complexity for auth checks where possible.

Prompt 5: Performance-Optimized API Endpoints (Progress Engine)**Plaintext**

Act as a Senior Backend Engineer. Context: PAPI Platform NestJS API.
Task: Implement the Progress Engine and Attendance APIs using modular services.

1. Write the 'AssessmentsService'. Create a method to bulk-insert assessment results.
2. Create a highly optimized query to fetch a student's progress over time, calculating averages.
3. Apply caching strategies (e.g., CacheModule) for static data like Recruitment Statistics.

Code must be lightning-fast and memory-efficient. Avoid N+1 queries using Prisma's 'select'.

Prompt 6: Frontend Dashboards & Data Visualization

Plaintext



Act as a Senior Frontend Engineer. Context: PAPI Next.js App.

Task: Build the Student Portal Dashboard and Charts.

1. Write a reusable React Hook (e.g., `useDataFetch`) wrapping SWR or React Query to
 2. Create a responsive `ProgressChart` component using Recharts that dynamically plo
 3. Implement the `StudentDashboard` page component using React Suspense and streaming
- Keep DOM nodes minimal

i want a fully ready srs pdf with detailed srs given above and the token efficient prompts

Here is the complete, self-contained **Software Requirements Specification (SRS) & Engineering Execution Blueprint**.

To turn this into a professional PDF immediately:

1. Open this page in your browser (Chrome/Edge/Firefox).
2. Press **Ctrl + P** (or **Cmd + P** on Mac).
3. Select **Destination: "Save as PDF"**, check **"Background graphics"**, set Margins to **Default**, and click **Save**.

The embedded CSS styling below is engineered specifically for print rendering, page breaks, typography, and clean code blocks.

HTML



```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>SRS - Purulia Aim Physical Institute (PAPI) Platform</title>
<style>
  @page {
    size: A4;
    margin: 20mm 15mm 20mm 15mm;
    @bottom-right {
      content: counter(page);
      font-size: 9pt;
      font-family: 'Segoe UI', system-ui, sans-serif;
      color: #666;
    }
  }

  body {
    font-family: 'Segoe UI', -apple-system, BlinkMacSystemFont, Roboto, Oxygen, Ubuntu;
    color: #1a1a1a;
    line-height: 1.45;
    font-size: 10pt;
    margin: 0;
    padding: 0;
  }

  /* Cover & Typography */
  h1, h2, h3, h4 {
    color: #26351F;
    font-weight: 700;
    margin-top: 1.2em;
    margin-bottom: 0.4em;
    page-break-after: avoid;
```

```
}  
  
h1 {  
  font-size: 20pt;  
  text-transform: uppercase;  
  letter-spacing: 0.5px;  
  border-bottom: 3px solid #26351F;  
  padding-bottom: 6px;  
  margin-top: 0;  
}  
  
h2 {  
  font-size: 13.5pt;  
  border-bottom: 1.5px solid #4B6135;  
  padding-bottom: 3px;  
}  
  
h3 {  
  font-size: 11pt;  
  color: #4B6135;  
}  
  
h4 {  
  font-size: 10pt;  
}  
  
p {  
  margin-top: 0;  
  margin-bottom: 0.6em;  
  text-align: justify;  
}  
  
/* Badges & Document Metadata */  
.meta-table {
```

```
width: 100%;
margin-bottom: 18px;
border-collapse: collapse;
}

.meta-table td {
padding: 5px 8px;
font-size: 9pt;
background: #F5F7F2;
border: 1px solid #d4dcce;
}

/* Tables */
table {
width: 100%;
border-collapse: collapse;
margin: 10px 0 14px 0;
font-size: 8.5pt;
page-break-inside: avoid;
}

th, td {
border: 1px solid #dcd8d8;
padding: 5.5px 7px;
text-align: left;
vertical-align: top;
}

th {
background-color: #26351F;
color: #ffffff;
font-weight: 600;
}
```

```
tr:nth-child(even) td {
  background-color: #fafbf9;
}

/* Highlights & Callouts */
.highlight-box {
  background-color: #F5F7F2;
  border-left: 4px solid #F59E0BAA;
  padding: 9px 12px;
  margin: 10px 0;
  font-size: 9pt;
}

/* Code & Token-Optimized Prompts */
pre {
  background: #0B0F0A;
  color: #E2E8F0;
  padding: 9px 11px;
  border-radius: 4px;
  font-family: 'Consolas', 'Fira Code', 'Courier New', monospace;
  font-size: 8pt;
  line-height: 1.35;
  white-space: pre-wrap;
  word-break: break-word;
  border-left: 4px solid #F59E0B;
  margin: 6px 0 12px 0;
  page-break-inside: avoid;
}

.prompt-header {
  font-weight: 700;
  color: #26351F;
  font-size: 9.5pt;
  margin-top: 14px;
```

```
margin-bottom: 2px;
display: flex;
justify-content: space-between;
}

.tag {
display: inline-block;
padding: 1px 5px;
border-radius: 3px;
font-size: 7.5pt;
font-weight: bold;
color: #fff;
background: #4B6135;
}

.page-break {
page-break-before: always;
}
</style>
</head>
<body>

<h1>Software Requirements Specification (SRS)</h1>
<table class="meta-table">
  <tr>
    <td><strong>System Name:</strong> Purulia Aim Physical Institute (PAPI) Platform</td>
    <td><strong>Version:</strong> 1.0 (Engineering Final)</td>
  </tr>
  <tr>
    <td><strong>Target Date:</strong> 28 September 2026</td>
    <td><strong>Status:</strong> Approved Architecture Draft</td>
  </tr>
  <tr>
    <td colspan="2"><strong>Scope:</strong> Public Web Portal + Training Management S
```

```
</tr>
</table>

<h2>1. Introduction & Strategic Vision</h2>
<p><strong>1.1 Purpose:</strong> This document establishes the formal functional, te

<p><strong>1.2 Background:</strong> Purulia Aim Physical Institute trains youth in pl

<p><strong>1.3 Product Vision:</strong> The software embodies three core principles:
  <strong>Discipline</strong> (military precision, structured metrics, immutable ver
  <strong>Energy</strong> (rapid loading, high-contrast visual cues, kinetic progres
  <strong>Trust</strong> (strict student data privacy, guardian consent governance,

<p><strong>1.4 Scope Boundaries:</strong>
  <ul>
    <li><strong>In-Scope (V1):</strong> SSR/SSG public site, Zod-validated multi-step
    <li><strong>Out-of-Scope (V1):</strong> Native mobile applications, real-time Wel
  </ul>
</p>

<h2>2. Operating Environment & User Matrix</h2>
<p><strong>2.1 Target Environment:</strong> Tier-2/3 network profiles (3G/4G bandwid

<table>
  <thead>
    <tr>
      <th style="width: 15%;">Role</th>
      <th style="width: 30%;">Access Boundary</th>
      <th style="width: 55%;">Responsibilities & Capabilities</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td><strong>Visitor</strong></td>
```

```

<td>Public Endpoints</td>
<td>Browses recruitment criteria, news, wall of fame; submits admissions applica
</tr>
<tr>
<td><strong>Applicant</strong></td>
<td>Unauthenticated State</td>
<td>Submitted application under review. No system access granted until verified
</tr>
<tr>
<td><strong>Student</strong></td>
<td>Self-Only (Row-Level Security)</td>
<td>Views personal attendance stats, physical progression charts, target gaps,
</tr>
<tr>
<td><strong>Trainer</strong></td>
<td>Assigned Batches Only</td>
<td>Creates daily sessions, executes single-transaction bulk attendance, record
</tr>
<tr>
<td><strong>Content Mgr</strong></td>
<td>Content Systems</td>
<td>Moderates student articles; maintains news, events, media gallery albums, a
</tr>
<tr>
<td><strong>Admin</strong></td>
<td>Institute Scope</td>
<td>Approves/rejects applications, assigns batches, manages recruitment criteri
</tr>
<tr>
<td><strong>Super Admin</strong></td>
<td>System Wide (Mandatory 2FA)</td>
<td>Configures RBAC matrices, rotates encryption keys, reviews immutable audit
</tr>
</tbody>

```

```
</table>
```

```
<h2>3. Functional Requirements Catalog</h2>
```

```
<h3>3.1 Public Portal (FR-PUB)</h3>
```

```
<ul>
```

- ```
 FR-PUB-01 (Homepage Core): Hero section with video/optimized
 FR-PUB-02 (Identity Assertion): Permanent display of core mis
 FR-PUB-03 (Program Directory): Program detail pages for Army,
 FR-PUB-04 (Recruitment Bulletin): Live board tracking status
 FR-PUB-05 (Wall of Fame & Gallery): Lightbox-driven categoriz
```

```

```

```
<h3>3.2 Application & Intake Pipeline (FR-APP)</h3>
```

```

```

- ```
  <li><strong>FR-APP-01 (Multi-Step Form):</strong> Intake sequence: Personal Teleme  
  <li><strong>FR-APP-02 (Consent Enforcement):</strong> Isolated checkboxes for Data  
  <li><strong>FR-APP-03 (Workflow Engine):</strong> State machine transitions: <code>
```

```
</ul>
```

```
<div class="page-break"></div>
```

```
<h3>3.3 Authentication, Security & RBAC (FR-AUTH)</h3>
```

```
<ul>
```

- ```
 FR-AUTH-01 (Session Architecture): Stateless JWT model with s
 FR-AUTH-02 (Cryptographic Storage): Argon2id configured with
 FR-AUTH-03 (Brute-Force & Abuse Mitigation): Sliding window r
 FR-AUTH-04 (Elevated 2FA): Time-based One-Time Password (TOTP)
```

```

```

```
<h3>3.4 Physical Telemetry & Progress Engine (FR-PRG)</h3>
```

```

```

- ```
  <li><strong>FR-PRG-01 (Structured Assessment Storage):</strong> Storage of metrics  
  <li><strong>FR-PRG-02 (Telemetry Processing):</strong> Deterministic API calculati
```

```
<li><strong>FR-PRG-03 (Comparative Visualizer):</strong> Vector-based progression</li></ul>

<div class="highlight-box">
  <strong>Illustrative Telemetry Progression Model:</strong>
  <table>
    <thead>
      <tr>
        <th>Assessment Variable</th>
        <th>Baseline (Intake)</th>
        <th>Current (30 Days)</th>
        <th>Target Benchmark (Army GD)</th>
        <th>Status Variance</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>1600m Endurance Run</td>
        <td>07m 42s (462s)</td>
        <td>06m 18s (378s)</td>
        <td>05m 45s (345s)</td>
        <td>-18.1% (On Target)</td>
      </tr>
      <tr>
        <td>Standard Push-ups</td>
        <td>22 reps</td>
        <td>41 reps</td>
        <td>50 reps</td>
        <td>+86.3% (Exceeding)</td>
      </tr>
      <tr>
        <td>Full Beam Pull-ups</td>
        <td>04 reps</td>
        <td>11 reps</td>
      </tr>
    </tbody>
  </table>
</div>
```

```
<td>10 reps</td>
<td><strong>Qualified (100%)</strong></td>
</tr>
<tr>
<td>Long Jump</td>
<td>3.10 m</td>
<td>3.65 m</td>
<td>4.25 m</td>
<td>Gap: 0.60 m</td>
</tr>
</tbody>
</table>
</div>

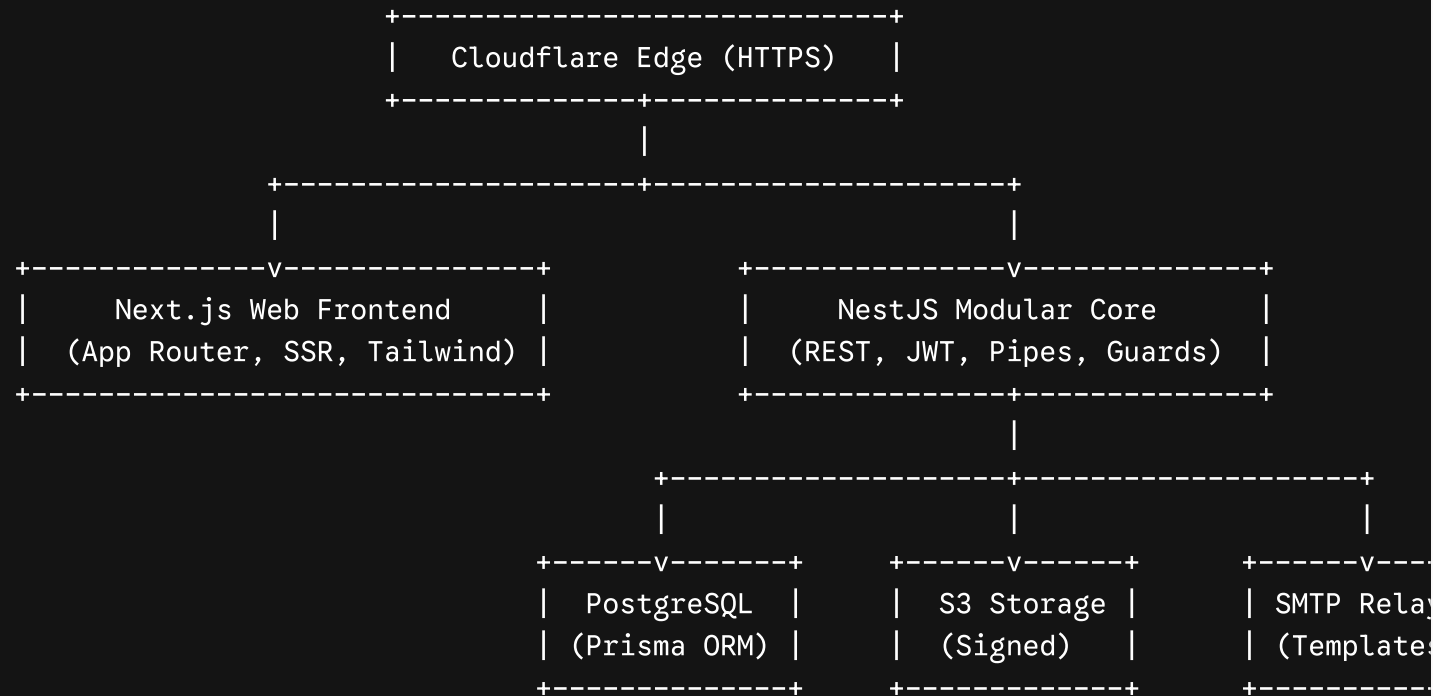
<h3>3.5 Training Sessions & Attendance Engine (FR-ATT)</h3>
<ul>
<li><strong>FR-ATT-01 (Session Provisioning):</strong> Trainers allocate session r
<li><strong>FR-ATT-02 (Transaction Batch Marking):</strong> Bulk attendance mutatio
<li><strong>FR-ATT-03 (Defaulter Detection):</strong> Automatic flagging of trained
</ul>

<h3>3.6 Content Management System (FR-CMS)</h3>
<ul>
<li><strong>FR-CMS-01 (Moderated Articles):</strong> Trainee journey submission pi
<li><strong>FR-CMS-02 (Wall of Fame Pipeline):</strong> Requires verified proof of
</ul>

<h2>4. Non-Functional & Quality Constraints</h2>
<ul>
<li><strong>Performance:</strong> Largest Contentful Paint (LCP) < 2.5s on mid-
<li><strong>Data Privacy:</strong> Zero plain-text persistence of sensitive identi
<li><strong>Reliability:</strong> 99.5% uptime target. Automated daily database du
<li><strong>Accessibility:</strong> WCAG 2.1 AA compliance across all dashboard in
</ul>
```

<h2>5. Technical Architecture & Monorepo Topology</h2>

```
<pre>
```



```
</pre>
```

```
<pre>
```

```
purulia-aim/
```

```

├── apps/
│   ├── web/                                # Next.js App Router (Client & Dashboards)
│   │   ├── app/
│   │   │   ├── (public)/                  # SSR Landing, Programs, Recruitments, Blog
│   │   │   ├── (auth)/                    # Secure cookie login, reset-password
│   │   │   ├── (portal)/student/          # Telemetry dashboards, targets, logs
│   │   │   ├── (portal)/trainer/          # Assessment entries, batch attendance
│   │   │   └── (portal)/admin/             # System control, audit, applications
│   │   └── components/ui/                 # Reusable generic primitives
│   └── api/                               # NestJS Backend API Application

```

```

├── src/
│   ├── auth/           # Passport, Argon2id, JWT Cookie Strategies
│   ├── telemetry/      # Physical assessment calculation engine
│   ├── attendance/     # Session batch transactions
│   ├── audit/          # Immutable audit log interceptors
│   ├── common/         # Global exception filters, guards, decorators
│   └── prisma/schema.prisma # Relational schema & indexes
└── packages/
    ├── types/          # Shared domain models & interfaces
    └── validation/      # Pure Zod schemas shared between client/server
</pre>

```

<div class="page-break"></div>

<h2>6. Relational Data Dictionary</h2>

Table Name	Primary / Foreign Keys	Indexed Attributes	Storage Purpose & Integrity Constraints
<code>users</code>	<code>id (UUID PK)</code>	<code>email, phone</code>	Authentication core. Password hashes, 2FA secret, account lock status.
<code>students</code>	<code>id (UUID PK), userId (FK)</code>		

```

<td><code>enrollmentNumber</code></td>
<td>Trainee demographics, guardian consent pointers, emergency contacts.</td>
</tr>
<tr>
<td><code>batches</code></td>
<td><code>id (UUID PK), trainerId (FK)</code></td>
<td><code>scheduleTime</code></td>
<td>Group management (e.g., Morning Alpha 05:30 AM).</td>
</tr>
<tr>
<td><code>training_sessions</code></td>
<td><code>id (UUID PK), batchId (FK)</code></td>
<td><code>sessionDate</code></td>
<td>Daily physical schedule details, workout micro-plans.</td>
</tr>
<tr>
<td><code>attendance</code></td>
<td><code>id (UUID PK), sessionId, studentId</code></td>
<td><code>[sessionId, studentId] (UQ)</code></td>
<td>Status (<code>PRESENT</code>, <code>ABSENT</code>, <code>LATE</code>, <code>...</code>)</td>
</tr>
<tr>
<td><code>assessments</code></td>
<td><code>id (UUID PK), batchId (FK)</code></td>
<td><code>date</code></td>
<td>Assessment parent definition (e.g., "Monthly 1600m & Strength Trial").</td>
</tr>
<tr>
<td><code>assessment_results</code></td>
<td><code>id (UUID PK), assessmentId, studentId</code></td>
<td><code>[assessmentId, studentId] (UQ)</code></td>
<td>Raw metrics: runTimeSeconds, pushups, situps, pullups, jumps.</td>
</tr>
<tr>

```

```

<td><code>target_standards</code></td>
<td><code>id (UUID PK)</code></td>
<td><code>forceName, category</code></td>
<td>Benchmark records (Army GD, WB Police SI, RPF Constable).</td>
</tr>
<tr>
<td><code>audit_logs</code></td>
<td><code>id (UUID PK), actorId (FK)</code></td>
<td><code>resource, createdAt</code></td>
<td>Immutable security trail (login, edits, permissions, deletions).</td>
</tr>
</tbody>
</table>

```

```
<hr style="border: 0; border-top: 2px solid #26351F; margin: 25px 0;">
```

```
<h1>Senior Engineering Execution Prompts</h1>
```

```
<p>These prompts are engineered to extract high-leverage code while conserving token
```

```
<div class="prompt-header">
```

```
<span>MODULE 1: MONOREPO SETUP, TURBOREPO & TYPESCRIPT FOUNDATION</span>
```

```
<span class="tag">EXECUTION PHASE 1</span>
```

```
</div>
```

```
<pre>
```

```
Act as a Principal Software Architect (15+ years experience).
```

```
Project: Purulia Aim Physical Institute (PAPI) Platform.
```

```
Stack: Turborepo, Next.js (App Router), NestJS, PostgreSQL, Prisma, Tailwind CSS, Zoc
```

```
TASK:
```

```
1. Provide the exact shell commands to scaffold a Turborepo monorepo with:
```

- `apps/web` (Next.js with Tailwind)
- `apps/api` (NestJS modular API)
- `packages/validation` (Shared Zod schemas)
- `packages/types` (Shared domain interfaces)

2. Provide root `package.json` with workspace configurations.
3. Configure `turbo.json` with optimized build, test, and lint dependency pipelines.
4. Output the root `tsconfig.base.json` enforcing strict null checks and module resolution.

RULES:

- Zero boilerplate narrative. Output raw scripts and production configuration code only.
- Strict DRY compliance.

</pre>

<div class="prompt-header">

MODULE 2: OPTIMIZED RELATIONAL PRISMA SCHEMA & SHARED CONTRACTS

EXECUTION PHASE 2

</div>

<pre>

Act as a Database Architect.

Context: PAPI Platform Postgres Schema via Prisma.

TASK:

1. Write the complete, production-grade `schema.prisma`.
 - Models: User, Role (Enum), Student, Batch, TrainingSession, Attendance, Assessment
 - Use UUIDv4 primary keys, explicit foreign key constraints with indexed lookup via Prisma
 - Enforce composite unique constraints: `@@unique([sessionId, studentId])` on Attendance
 - Use smallint and integer data types for metrics (e.g., `runTimeSeconds`, `pushupsCompleted`)
2. In `packages/validation`, write the reusable Zod validation schemas for:
 - Trainee admission application (multi-step validation including minor guardian consent)
 - Single-transaction bulk assessment recording.
3. Export inferred TypeScript types directly from Zod definitions.

RULES:

- No placeholder comments (`// add more fields here`). Write the full schema.
- Include database-level indexing for high-frequency queries.

</pre>

<div class="page-break"></div>

```
<div class="prompt-header">
  <span>MODULE 3: HIGH-PERFORMANCE BACKEND CORE, AUTH & RBAC</span>
  <span class="tag">EXECUTION PHASE 3</span>
</div>
<pre>
Act as a Principal Backend Security Engineer.
Context: NestJS API application in `apps/api`.

TASK:
Write the complete, memory-efficient authentication and authorization subsystem:
1. `AuthService`:
  - Password hashing using `argon2id` (memoryCost: 65536, timeCost: 3).
  - Login issuing a short-lived Access Token (15m) and rotating Refresh Token (7d).
  - Write cryptographic tokens directly to secure cookies: `HttpOnly; Secure; SameSite=Strict`.
2. `JwtAuthGuard` & `RolesGuard`:
  - Reflective execution reading role requirements via custom `@Roles(...)` metadata.
  - Extraction of JWT from signed cookies (zero Authorization header exposure).
3. `AuditInterceptor`:
  - Interceptor logging mutation events (POST, PATCH, DELETE) to the `AuditLog` table.
4. Generic NestJS Exception Filter sanitizing raw database errors and returning standard responses.

RULES:
- Pure TypeScript. Avoid unnecessary imports. Keep memory footprint minimal.
</pre>

<div class="prompt-header">
  <span>MODULE 4: QUANTITATIVE TELEMETRY ENGINE & BATCH ATTENDANCE</span>
  <span class="tag">EXECUTION PHASE 4</span>
</div>
<pre>
Act as a Senior Backend Systems Developer.
Context: NestJS Telemetry & Attendance Modules.
```

TASK:

1. Implement `AttendanceService`:

- Method `recordBulkAttendance(sessionId: string, records: { studentId: string, s`
- Execute in a single atomic database `\$transaction`.
- Flag trainees breaching 3 consecutive absences without separate read queries.

2. Implement `ProgressEngineService`:

- Method `calculateStudentTelemetry(studentId: string, targetForce: Force)`:
- Query last 5 assessments and compute: BMI, % change for 1600m run, push-ups, and
- Fetch benchmark criteria for the selected force from `TargetStandard` and calcul
- `{ metric, current, target, qualified: boolean, gap: number }`.
- Prevent N+1 queries by using Prisma's `select` projection.

RULES:

- Production-ready code with complete logic. No partial implementations.

</pre>

<div class="prompt-header">

MODULE 5: GENERIC, ACCESSIBLE REUSABLE FRONTEND PRIMITIVES

EXECUTION PHASE 5

</div>

<pre>

Act as a Senior Frontend UI Architect.

Context: Next.js App Router (`apps/web`), Tailwind CSS, Radix UI.

TASK:

Build completely generic, fully-typed UI primitives honoring the PAPI brand design to

Primary: Deep Olive (#26351F), Secondary: Army Green (#4B6135), Accent: Saffron (#F59E00)

1. `DataTable<TData, TValue>`:

- Headless table wrapper powered by TanStack Table.
- Virtualized rendering for performance with hundreds of student records.
- Dynamic sorting, filtering, and pagination support.

2. `FormWrapper`:

- Generic form engine integrating `react-hook-form` and shared Zod schemas.

- Auto-generates field errors and loading states.
3. `StatMetricCard`:
- Displays performance KPIs: current value, baseline, percentage improvement pill

RULES:

- Zero `any` types. Provide full TypeScript generics.
- No code duplication across dashboard components.

</pre>

```
<div class="prompt-header">  
  <span>MODULE 6: STUDENT PROGRESS VISUALIZER & REPORT EXPORT</span>  
  <span class="tag">EXECUTION PHASE 6</span>  
</div>
```

<pre>

Act as a Senior Full-Stack Engineer.

Context: Next.js (`apps/web`) Trainee Telemetry Portal.

TASK:

1. Build `StudentProgressVisualizer`:
 - Responsive multi-metric chart using Recharts.
 - Plot 1600m running times over date sequences with a horizontal reference line for baseline.
 - Tooltip formatting displaying absolute improvement in seconds and percentage change.
2. Build `ProgressReportPDFView`:
 - Print-ready and CSS-isolated evaluation sheet component containing student stats.
3. Optimize page performance to ensure Largest Contentful Paint (LCP) < 2.5s using

RULES:

- Mobile-first responsiveness (360px viewport upwards).
- Clean code architecture with clear visual separation.

</pre>